Chunking & Retrieval

How the NavAI corpus is split, indexed, fused, and reranked

01 CHUNK SIZE

Nine DOCX documents were split into fixed token windows with a 32-token overlap. Three sizes were compared with retrieval and reranking held fixed. Chunk and rerank scores below use the controlled trials.

Size	Chunks	Correct
64	407	31/59
128	133	34/59
256	57	37/59

Average context: 337 / 586 / 1,091 tokens. The 256-token setting also fits the embedding model's 256-token input limit, so no chunk is truncated.

Why 256

It gave the highest accuracy at fixed retrieval settings, kept average context near 1.1k tokens, and avoids truncation. The 4-vs-8 rerank comparison is below.

02 RETRIEVER

Vector store	ChromaDB, persistent local index, cosine HNSW.
Collections	One collection per chunk size.
Embedding	all-MiniLM-L6-v2.
Dense	Top 20 chunks.
BM25	In-memory from Chroma; used with dense.
Fusion	Reciprocal rank fusion; combines both rankings.
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2; top 4.

BM25 uses no extra service or dependency. The cross-encoder reranks the fused candidates.

Retrieval flow

Rerank depth: top 4 vs top 8

Chunk	Top 4	Top 8	Avg context 4 / 8
64	31/59	32/59	337 / 669
128	34/59	33/59	586 / 1,170
256	37/59	38/59	1,091 / 2,174

At 256, top 8 added one correct case but raised average context from 1,091 to 2,174 tokens and total input from 78,102 to 141,989. It added one case at 64 and lost one at 128. Top 4 was retained.

03 BACKEND STRUCTURE

Offline: DOCX → fixed chunks → embeddings → ChromaDB.

Online: browser / CLI → FastAPI /api/chat → NavAIChat → dense + BM25 → RRF → cross-encoder → LLM → answer + sources.

app.py API · pipeline.py orchestration · retrieval.py hybrid retrieval · prompts.py / answers.py prompt + citations · settings.py config

BM25 + RRF did not change the score much.